

Инъекция команд ОС

Описание проблемы

Инъекция команд ОС – это уязвимость, которая возникает из-за того, что входные данные не валидируются, и после этого попадают в функции/методы, выполняющие команды в операционной системе. По сути, благодаря такой уязвимости, злоумышленник может непредсказуемо изменять поведение программы и расширять ее функциональность без ведома разработчика.

В Python выполнить команды ОС можно с помощью:

- методов модуля **subprocess**: Popen, run, call, check_call, check_output, getstatusoutput
- методов модуля **cmd**: cmd, prompt, postcmd, precmd, default, cmdloop
- метода модуля **commands**: getstatusoutput
- метода модуля **fabric**: run
- методы модуля **os**: popen, system, popen2, popen3, popen4, spawn*
- метода модуля **pexpect**: spawn
- метода модуля **pty**: spawn
- методы модуля **pyexec**: install, exec
- методы модуля **sh**: Command, sleep, run
- метода модуля **socket**: bind
- экземпляр объекта типа **stdout** (__class__)
- метода модуля **asyncio**: create_subprocess_exec
- метода модуля **paramiko**: exec_command

Последствия эксплуатации

Представьте, что злоумышленник может делать все тоже самое, что и ваш системный администратор. Или, как минимум, доверенный пользователь системы с доступом к командной оболочке сервера. Уже можно примерно понять потенциальный ущерб от такой возможности: утеря и модификация данных системы, вывод приложения из строя. А чаще всего инъекция команды ОС используется злоумышленниками, чтобы получить доступ к другим частям инфраструктуры организации.

Пример небезопасного кода

С использованием модуля os:

```
1 user_input = params.Get('SearchString')
2 os.spawnlpe(os.P_WAIT, user_input, ["-a"], os.environ)
```

С использованием модуля subprocess:

```
1 user_input = params.Get('SearchString')
2 subprocess.call("grep -R {} {}".format(user_input), shell=True)
```

Конкатенация строк при формировании команд ОС – **всегда** плохая идея:

```
1 user_input = params.Get('SearchString')
2 subprocess.run(["bash", "-c", "myCommand myArg1 " + user_input])
```

Пользователь может передать в параметре "foo && cat /etc/passwd" и узнать пароли.

Как исправить?

Прежде всего убедитесь, что вам действительно необходимы входные данные при формировании команд операционной системы. Часто логику приложения можно изменить таким образом, чтобы пользовательские данные в такие критичные функции не попадали.

Если это не удастся и динамическое формирование команды действительно необходимо, то необходимо передавать команду в метод, выполняющий команду ОС, в виде массива аргументов. Например, это позволяет делать модуль `subprocess`.



Не используйте библиотеки и методы, которые не позволяют передавать команду ОС в виде массива аргументов!

Пример безопасного кода

```
1 user_input = params.Get('SearchString')
2 subprocess.Popen(['ls', '-l', user_input])
```

Если в данном случае передать `"foo && cat /etc/passwd"`, то пароли злоумышленнику узнать не получится: команда не выполнится.



Важно!

Не используйте `shell=True` даже если передаете параметры в метод в виде массива аргументов. Этот аргумент позволяет определить методу, нужно ли воспринимать входную строку как полноценную команду, что повлечет ее выполнение, даже если используется массив аргументов. По умолчанию у аргумента `shell` значение `"False"`.

```
1 user_input = params.Get('SearchString')
2 subprocess.run(["bash", "-c", user_input], shell=True)
```

Альтернативные способы

Если приведенный выше вариант исправления инъекции командной строки невозможно реализовать в вашем контексте, то можно использовать "белый список".

Сначала вы должны составить ограниченный список команд, который предполагаете будет выполнять ваш код. Затем проверять, что входные данные (либо значение, которое формировалось при помощи входных данных) содержатся в этом списке. Если нет - передавать их дальше на выполнение нельзя.

Пример:

```
1 whitelist = ('value1', 'value2', 'value3', 'value4')
2 user_input = params.Get('SearchString')
3
4 if user_input in whitelist:
5     subprocess.call("grep -R {} {}".format(user_input), shell=True)
```

Здесь злоумышленнику не получится исполнить произвольную команду, даже если он передаст соответствующее значение. Команда просто не выполнится, потому что ее нет в нашем "белом списке". Поэтому важно подойти к составлению "белого списка" со всей строгостью.

Полезные ссылки

1. Инъекция команд ОС в [OWASP](#)
2. Атрибут `shell` в [subprocess](#) - документация Python